

So far, we have looked at a version of Hoare logic that supports reasoning about IMP programs in terms of pre- and post-conditions on stores. In this lecture, we consider a language with pointers and dynamic memory allocation. We define a big-step operational semantics for the language as a relation on program states (σ, h) with a store and a heap. We show how Hoare logic fails to support modular reasoning due to aliasing between pointers. Finally, we introduce *separation logic*, which extends Hoare logic's assertions with predicates on heaps and provides a *frame* rule that supports modular proofs.

1 IMP with Heaps

As a first step, we extend IMP with standard commands for manipulating values on the heap. Note: in this lecture we will write e for arithmetic expressions rather than a , as it is more intuitive:

$$c ::= \text{skip} \mid x := e \mid c_1; c_2 \mid \text{if } b \text{ then } c_1 \text{ else } c_2 \mid \text{while } b \text{ do } c \\ \mid x := \text{new}(e) \mid \text{free}(e) \mid x := *e \mid *e_1 := e_2$$

The new commands in this language include:

- $x := \text{new}(e)$, which evaluates e to a value v , allocates storage for v on the heap, and assigns the pointer to that storage to x in the store;
- $x := *e$, which evaluates e to a pointer p , loads the value v associated with p from the heap, and assigns v to x in the store;
- $*e_1 := e_2$, which evaluates e_1 to a pointer p and e_2 to a value v , and then stores v at the location associated with p on the heap; and
- $\text{free}(e)$, which evaluates e to a pointer p and deallocates the heap storage associated with p .

2 Big-Step Operational Semantics

To define the semantics of this extension of IMP, we will need states that keep track both of the local variables as well as the heap. We will model states as pairs (σ, h) where

- $\sigma \in \text{Var} \rightarrow \mathbb{Z}$ is the store, which for simplicity we take to be a total function, and
- $h \in \text{Addr} \rightarrow_{\text{fin}} \mathbb{Z}$ is the heap, a finite partial function.

Here $\text{Addr} \subseteq \mathbb{Z} \setminus \{0\}$ is the set of addresses. Pointers are just integers, so they can be stored in variables, stored on the heap, and computed with arithmetic expressions. We exclude 0 so that it can serve as the null pointer: no cell is ever stored at address 0. As usual, we write $\text{dom}(h)$ for the domain of the heap and $h[p \mapsto v]$ for the heap that maps p to v and otherwise behaves like h .

In the formal semantics, commands evaluate in one big step to an *outcome* ω , which is either a final state or a memory error:

$$\langle \sigma, h, c \rangle \Downarrow \omega \quad \omega ::= (\sigma', h') \mid \mathbf{err}$$

while arithmetic and boolean expressions remain pure,

$$\langle \sigma, e \rangle \Downarrow v$$

Basic commands

$$\begin{array}{c} \frac{}{\langle \sigma, h, \mathbf{skip} \rangle \Downarrow (\sigma, h)} \text{SKIP} \quad \frac{\langle \sigma, e \rangle \Downarrow v}{\langle \sigma, h, x := e \rangle \Downarrow (\sigma[x \mapsto v], h)} \text{ASSIGN} \\[10pt] \frac{\langle \sigma, h, c_1 \rangle \Downarrow (\sigma_1, h_1) \quad \langle \sigma_1, h_1, c_2 \rangle \Downarrow \omega}{\langle \sigma, h, c_1; c_2 \rangle \Downarrow \omega} \text{SEQ} \end{array}$$

Conditionals and loops

$$\begin{array}{c} \frac{\langle \sigma, b \rangle \Downarrow \mathbf{true} \quad \langle \sigma, h, c_1 \rangle \Downarrow \omega}{\langle \sigma, h, \mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2 \rangle \Downarrow \omega} \text{IF-TRUE} \quad \frac{\langle \sigma, b \rangle \Downarrow \mathbf{false} \quad \langle \sigma, h, c_2 \rangle \Downarrow \omega}{\langle \sigma, h, \mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2 \rangle \Downarrow \omega} \text{IF-FALSE} \\[10pt] \frac{\langle \sigma, b \rangle \Downarrow \mathbf{false}}{\langle \sigma, h, \mathbf{while } b \mathbf{ do } c \rangle \Downarrow (\sigma, h)} \text{WHILE-FALSE} \\[10pt] \frac{\langle \sigma, b \rangle \Downarrow \mathbf{true} \quad \langle \sigma, h, c \rangle \Downarrow (\sigma_1, h_1) \quad \langle \sigma_1, h_1, \mathbf{while } b \mathbf{ do } c \rangle \Downarrow \omega}{\langle \sigma, h, \mathbf{while } b \mathbf{ do } c \rangle \Downarrow \omega} \text{WHILE-TRUE} \end{array}$$

Heap commands

$$\begin{array}{c} \frac{\langle \sigma, e \rangle \Downarrow p \quad p \in \text{dom}(h) \quad h(p) = v}{\langle \sigma, h, x := *e \rangle \Downarrow (\sigma[x \mapsto v], h)} \text{LOAD} \quad \frac{\langle \sigma, e_1 \rangle \Downarrow p \quad \langle \sigma, e_2 \rangle \Downarrow v \quad p \in \text{dom}(h)}{\langle \sigma, h, *e_1 := e_2 \rangle \Downarrow (\sigma, h[p \mapsto v])} \text{STORE} \\[10pt] \frac{\langle \sigma, e \rangle \Downarrow v \quad p \in \text{Addr} \setminus \text{dom}(h)}{\langle \sigma, h, x := \mathbf{new}(e) \rangle \Downarrow (\sigma[x \mapsto p], h[p \mapsto v])} \text{NEW} \quad \frac{\langle \sigma, e \rangle \Downarrow p \quad p \in \text{dom}(h)}{\langle \sigma, h, \mathbf{free}(e) \rangle \Downarrow (\sigma, h|_{\text{dom}(h) \setminus \{p\}})} \text{FREE} \end{array}$$

Here $h|_A$ is the restriction of h to the addresses in A . Note that NEW may choose *any* unused address p , so the semantics is nondeterministic.

Memory errors A load, store, or free at an address that is not allocated produces a memory error, and errors propagate through sequencing and loops:

$$\begin{array}{c}
\frac{\langle \sigma, e \rangle \Downarrow p \quad p \notin \text{dom}(h)}{\langle \sigma, h, x := *e \rangle \Downarrow \mathbf{err}} \text{LOAD-ERR} \qquad \frac{\langle \sigma, e_1 \rangle \Downarrow p \quad p \notin \text{dom}(h)}{\langle \sigma, h, *e_1 := e_2 \rangle \Downarrow \mathbf{err}} \text{STORE-ERR} \\
\\
\frac{\langle \sigma, e \rangle \Downarrow p \quad p \notin \text{dom}(h)}{\langle \sigma, h, \mathbf{free}(e) \rangle \Downarrow \mathbf{err}} \text{FREE-ERR} \qquad \frac{\langle \sigma, h, c_1 \rangle \Downarrow \mathbf{err}}{\langle \sigma, h, c_1; c_2 \rangle \Downarrow \mathbf{err}} \text{SEQ-ERR} \\
\\
\frac{\langle \sigma, b \rangle \Downarrow \mathbf{true} \quad \langle \sigma, h, c \rangle \Downarrow \mathbf{err}}{\langle \sigma, h, \mathbf{while } b \text{ do } c \rangle \Downarrow \mathbf{err}} \text{WHILE-ERR}
\end{array}$$

The rules for SEQ, IF, and WHILE-TRUE above already pass along an error from their last premise. Since $0 \notin \text{Addr}$, any load, store, or free at address 0 produces **err**. No other command can fail: the store is total, so expressions always evaluate, and we assume that Addr is infinite, so NEW can always find an unused address in the finite heap. Thanks to the outcome **err**, a program that crashes is now distinguishable from one that runs forever, which has no derivation at all.

3 The Rule of Constancy

The following rule is admissible in standard Hoare logic:

$$\frac{\vdash \{P\} c \{Q\} \quad fv(R) \cap \text{mod}(c) = \emptyset}{\vdash \{P \wedge R\} c \{Q \wedge R\}} \text{CONSTANCY}$$

Here $fv(R)$ are the free variables of R and $\text{mod}(c)$ are the variables that c may modify. For example, $fv(y = 0) = \{y\}$ and $\text{mod}(x := x+1) = \{x\}$.

Intuitively, the rule of constancy captures a form of local reasoning. It allows us to first prove a simple Hoare triple and then extend it to a more complicated triple by conjoining the same predicate to the pre- and post-conditions. For instance, using this rule we can strengthen

$$\{x > 0\} x := x+1 \{x > 1\}$$

to

$$\{x > 0 \wedge y = 0\} x := x+1 \{x > 1 \wedge y = 0\}$$

Free and Modified Variables

The function $fv(P)$ returns the set of program variables that occur free in assertion P .

$$\begin{aligned}
fv(n) &= fv(\mathbf{true}) = fv(\mathbf{false}) &= \emptyset \\
fv(x) &= \{x\} \\
fv(a_1 + a_2) &= fv(a_1 \times a_2) = fv(a_1 \leq a_2) &= fv(a_1) \cup fv(a_2) \\
fv(P \wedge Q) &= fv(P \vee Q) = fv(P \Rightarrow Q) &= fv(P) \cup fv(Q) \\
fv(\neg P) &= fv(\forall i. P) = fv(\exists i. P) &= fv(P)
\end{aligned}$$

The quantifier cases do not remove anything, because i is a logical variable, not a program variable. The function $mod(c)$ returns the set of program variables modified by command c .

$$\begin{aligned} mod(\mathbf{skip}) &= mod(*e_1 := e_2) = mod(\mathbf{free}(e)) &= \emptyset \\ mod(x := e) &= mod(x := *e) = mod(x := \mathbf{new}(e)) &= \{x\} \\ mod(c_1; c_2) &= mod(\mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2) &= mod(c_1) \cup mod(c_2) \\ mod(\mathbf{while } b \mathbf{ do } c) &= mod(c) \end{aligned}$$

Issues Related to Aliasing Unfortunately, the rule of constancy is not sound when we extend our language of assertions with heap predicates. For example, suppose that we add an assertion $e_1 \hookrightarrow e_2$, which says that the heap *contains* a cell at address e_1 holding e_2 (and possibly other cells as well). Now consider trying to use the rule of constancy to add a “constant fact” about the heap. We might first prove the following triple:

$$\{x \hookrightarrow -\} *x := 4 \{x \hookrightarrow 4\}.$$

and then use the rule of constancy to obtain:

$$\{x \hookrightarrow - \wedge y \hookrightarrow 3\} *x := 4 \{x \hookrightarrow 4 \wedge y \hookrightarrow 3\}.$$

Here $mod(*x := 4) = \emptyset$, so the side condition of the rule is satisfied trivially. However, if x and y are aliases for each other, then the postcondition is false. Of course, it is possible to complicate the pre- and post-condition to capture disjointness predicates, but this quickly becomes impractical—in principle, every pointer on the heap might be aliased to every other pointer, which leads to a combinatorial explosion. Such complications motivated the development of separation logic.

4 Heap Assertions

Before defining separation logic, let us extend our language of assertions with heap predicates:

$$H, J, K ::= \mathbf{emp} \mid [P] \mid e_1 \hookrightarrow e_2 \mid H_1 \star H_2 \mid H_1 \frown H_2 \mid H_1 \wp H_2 \mid \forall i. H \mid \exists i. H$$

Intuitively, these predicates can be understood as follows.

- $\mathbf{emp}$: heap is empty,
- $[P]$: heap is empty and P holds,
- $e_1 \hookrightarrow e_2$: heap consists of *exactly* one cell with pointer e_1 and value e_2 (this is stricter than the informal reading of $\hookrightarrow$ in the previous section),
- $H_1 \star H_2$: heap splits into two disjoint pieces, one satisfying H_1 and the other H_2 ,
- $H_1 \frown H_2$ and $H_1 \wp H_2$ are heap assertion versions of the standard boolean connectives, and
- $\forall i. H$ and $\exists i. H$ are heap assertion versions of the standard quantifiers.

More formally, we can model their semantics as follows:

$$\begin{array}{ll}
(\sigma, h) \models_I \mathbf{emp} & \text{if } h = \emptyset \\
(\sigma, h) \models_I [P] & \text{if } h = \emptyset \wedge \sigma \models_I P \\
(\sigma, h) \models_I e_1 \hookrightarrow e_2 & \text{if } h = \{(p, v)\} \text{ where } \langle \sigma, e_1 \rangle_I \Downarrow p \text{ and } \langle \sigma, e_2 \rangle_I \Downarrow v \\
(\sigma, h) \models_I H_1 \star H_2 & \text{if } \exists h_1, h_2. h = h_1 \uplus h_2 \wedge (\sigma, h_1) \models_I H_1 \wedge (\sigma, h_2) \models_I H_2 \\
(\sigma, h) \models_I H_1 \wp H_2 & \text{if } (\sigma, h) \models_I H_1 \text{ and } (\sigma, h) \models_I H_2 \\
(\sigma, h) \models_I H_1 \vee H_2 & \text{if } (\sigma, h) \models_I H_1 \text{ or } (\sigma, h) \models_I H_2 \\
(\sigma, h) \models_I \forall i. H & \text{if } (\sigma, h) \models_{I[i \mapsto v]} H \text{ for all } v \\
(\sigma, h) \models_I \exists i. H & \text{if } (\sigma, h) \models_{I[i \mapsto v]} H \text{ for some } v
\end{array}$$

Here $h_1 \uplus h_2$ is the union of heaps h_1 and h_2 with disjoint domains (it is undefined if they overlap). Free variables extend to heap assertions in the obvious way, e.g., $fv(e_1 \hookrightarrow e_2) = fv(e_1) \cup fv(e_2)$, $fv(\mathbf{emp}) = \emptyset$, and $fv([P]) = fv(P)$.

Quantifiers bind *logical* variables (never program variables), which range over integers, including addresses, and over finite lists of integers; I maps each to a value of the right kind. We typically write i, j, a, b for integers and L for lists.

We will often abbreviate $e \hookrightarrow - \triangleq \exists i. e \hookrightarrow i$.

As heaps only contain addresses, $0 \hookrightarrow v$ is unsatisfiable for every v : there is no heap whose single cell is at address 0. Similarly, predicates like $x \hookrightarrow 5 \star x \hookrightarrow 5$ are *unsatisfiable* as the separating conjunction operator, $\star$, enforces disjointness—even when the two cells would hold the same value.

Equivalence and entailment. Two heap assertions are *equivalent*, written $H_1 \equiv H_2$, if they hold in exactly the same states:

$$H_1 \equiv H_2 \triangleq \forall \sigma, h, I. (\sigma, h) \models_I H_1 \iff (\sigma, h) \models_I H_2.$$

(Note that it is *not* enough for H_1 and H_2 to be both valid or both invalid—for example, $x \hookrightarrow 1$ and $x \hookrightarrow 2$ are both invalid, but they are certainly not equivalent.) Similarly, H_1 *entails* H_2 , written $H_1 \Rightarrow H_2$, if every state satisfying H_1 also satisfies H_2 .

Separating conjunction satisfies several familiar algebraic laws:

$$\begin{array}{ll}
H \star J & \equiv J \star H & \text{(commutativity)} \\
(H \star J) \star K & \equiv H \star (J \star K) & \text{(associativity)} \\
H \star \mathbf{emp} & \equiv H & \text{(unit)}
\end{array}$$

However, $\star$ and $\wedge$ behave differently. In general, $[P] \star H \not\equiv [P] \wp H$: the left-hand side says that P holds and H describes the whole heap, while the right-hand side requires the heap to be empty. For instance, $[\mathbf{true}] \star x \hookrightarrow 1$ is satisfiable, but $[\mathbf{true}] \wp x \hookrightarrow 1$ is not. When H is $\mathbf{emp}$, the two do coincide: $[P] \star \mathbf{emp} \equiv [P] \wp \mathbf{emp}$. In practice, $[P] \star H$ is the idiomatic way to state a pure fact P alongside a heap assertion H , and we will use entailments such as $[P] \star H \Rightarrow H$ freely.

5 Valid Triples

Now we define the notion of valid triples. We use the standard definition for separation logic, which is *partial correctness with fault avoidance*, following Reynolds and O'Hearn: a valid triple guarantees that the program never produces a memory error, and that *if* it terminates, the postcondition

holds. The explicit outcome **err** is what makes this possible. If memory errors simply got stuck, a crashing program would have no derivation, just like a program that runs forever, and a partial correctness triple would hold vacuously for it. With fault avoidance, $\{\mathbf{emp}\} \text{ free}(x) \{\mathbf{false}\}$ is *not* valid, because $\text{free}(x)$ produces **err** on the empty heap. On the other hand, $\{\mathbf{emp}\} \text{ while true do skip } \{\mathbf{false}\}$ is valid: the loop never crashes and never terminates. As usual for partial correctness, termination must be proved separately if it is needed.

Definition (Valid Triple). A triple is valid, written $\models \{H\} c \{J\}$, if for all σ, h , and I with $\sigma, h \models_I H$:

- $\langle \sigma, h, c \rangle \Downarrow \mathbf{err}$ is not derivable, and
- every σ', h' with $\langle \sigma, h, c \rangle \Downarrow (\sigma', h')$ satisfies $\sigma', h' \models_I J$.

Because **NEW** makes the semantics nondeterministic, both conditions quantify over *all* executions: a valid triple rules out an error for every possible choice of fresh addresses.

Note that the precondition H describes the *entire* heap in which c runs, and c must not produce an error in any such heap. So a valid triple implicitly says that c only accesses cells described by H —its *footprint*. For example, $\{x \hookrightarrow 3\} *x := 4 \{x \hookrightarrow 4\}$ is valid, but $\{\mathbf{emp}\} *x := 4 \{\mathbf{emp}\}$ is not, because $*x := 4$ produces **err** on the empty heap.

6 The Frame Rule

A major appeal of separation logic is that it supports the so-called frame rule:

$$\frac{\vdash \{H\} c \{J\} \quad fv(K) \cap mod(c) = \emptyset}{\vdash \{H \star K\} c \{J \star K\}} \text{FRAME}$$

The rule captures modular reasoning about the heap. Intuitively, we first prove a specification for c using only the “heap” footprint it needs. Then the command can be framed in any larger context obtained by combining the footprint with a disjoint heap.

Soundness. Why is the frame rule sound? Write $h \perp h_0$ when h and h_0 have disjoint domains. The proof rests on four facts, the first three about the semantics (each proved by induction on derivations) and the last about assertions (proved by induction on the assertion):

- *Safety monotonicity*: if $\langle \sigma, h, c \rangle \Downarrow \mathbf{err}$ is not derivable, then neither is $\langle \sigma, h \uplus h_0, c \rangle \Downarrow \mathbf{err}$, for any $h_0 \perp h$. (If a load, store, or free finds its address in h , it also finds it in $h \uplus h_0$.)
- *Frame property*: if $\langle \sigma, h, c \rangle \Downarrow \mathbf{err}$ is not derivable and $\langle \sigma, h \uplus h_0, c \rangle \Downarrow (\sigma', h')$, then $h' = h'_1 \uplus h_0$ for some h'_1 with $\langle \sigma, h, c \rangle \Downarrow (\sigma', h'_1)$. In other words, a run in the larger heap leaves h_0 untouched and behaves like a run in h .
- *Unmodified variables*: if $\langle \sigma, h, c \rangle \Downarrow (\sigma', h')$, then $\sigma'(y) = \sigma(y)$ for every $y \notin mod(c)$.
- *Coincidence* (also called the agreement or relevance lemma): if σ and σ' agree on $fv(K)$, then $\sigma, h \models_I K$ if and only if $\sigma', h \models_I K$. That is, an assertion depends on the store only through its free variables.

The interesting case of the frame property is **NEW**. The run in $h \uplus h_0$ allocates some $p \notin \text{dom}(h \uplus h_0)$. Since $\text{dom}(h) \subseteq \text{dom}(h \uplus h_0)$, this p is also unused in h , so the run in h alone can pick the *same* p , and the results match: $(h \uplus h_0)[p \mapsto v] = h[p \mapsto v] \uplus h_0$. This is exactly why **NEW** is allowed to choose *any* unused address. With a deterministic allocator, say $p = 1 + \max \text{dom}(h)$, the two runs could allocate different addresses, and the frame rule would be unsound: on the empty heap the allocator returns 1, so $\{\mathbf{emp}\} x := \mathbf{new}(1) \{[x = 1] \star x \hookrightarrow 1\}$ would be valid, but framing in $1 \hookrightarrow 5$ gives a false triple, because the allocator would now return 2.

Theorem (Soundness of Frame). *If $\models \{H\} c \{J\}$ and $\text{fv}(K) \cap \text{mod}(c) = \emptyset$, then $\models \{H \star K\} c \{J \star K\}$.*

Proof. Suppose $\sigma, h \models_I H \star K$. Then $h = h_1 \uplus h_0$ with $\sigma, h_1 \models_I H$ and $\sigma, h_0 \models_I K$. By the premise, $\langle \sigma, h_1, c \rangle \Downarrow \mathbf{err}$ is not derivable, so by safety monotonicity neither is $\langle \sigma, h, c \rangle \Downarrow \mathbf{err}$. Now suppose $\langle \sigma, h, c \rangle \Downarrow (\sigma', h')$. By the frame property, $h' = h'_1 \uplus h_0$ with $\langle \sigma, h_1, c \rangle \Downarrow (\sigma', h'_1)$, so $\sigma', h'_1 \models_I J$ by the premise. By the unmodified-variables fact, σ' agrees with σ on $\text{fv}(K)$, since $\text{fv}(K) \cap \text{mod}(c) = \emptyset$; so by coincidence we still have $\sigma', h_0 \models_I K$. Hence $\sigma', h' \models_I J \star K$. $\square$

Why fault avoidance? The proof uses the absence of errors in an essential way. Suppose instead that memory errors simply got stuck, and triples were ordinary partial correctness statements. Then $\{\mathbf{emp}\} \mathbf{free}(x) \{\mathbf{emp}\}$ would be valid vacuously, since $\mathbf{free}(x)$ has no result on the empty heap. But framing in $x \hookrightarrow 3$ would give $\{x \hookrightarrow 3\} \mathbf{free}(x) \{x \hookrightarrow 3\}$, which is false: the free succeeds and the cell is gone. Fault avoidance rules out the first triple, because it forces a command to own every cell it touches.

Baking in the frame rule. Some modern presentations, such as Charguéraud's, take a different approach and build the frame rule into the definition of triples. There, $\{H\} c \{J\}$ is defined to be valid if, for every K with $\text{fv}(K) \cap \text{mod}(c) = \emptyset$, the triple $\{H \star K\} c \{J \star K\}$ is valid in the sense above. The frame rule is then sound by construction, at the cost of a less direct definition.

7 Small Axioms for Heap Commands

We can define “small” axioms for the heap-manipulating commands as follows.

$$\begin{array}{c}
 \frac{x \notin \text{fv}(e_1) \cup \text{fv}(e_2)}{\{e_1 \hookrightarrow e_2\} x := *e_1 \{[x = e_2] \star e_1 \hookrightarrow e_2\}} \text{LOAD} \\
 \\
 \frac{}{\{e_1 \hookrightarrow -\} *e_1 := e_2 \{e_1 \hookrightarrow e_2\}} \text{STORE} \\
 \\
 \frac{x \notin \text{fv}(e)}{\{\mathbf{emp}\} x := \mathbf{new}(e) \{x \hookrightarrow e\}} \text{ALLOC} \\
 \\
 \frac{}{\{e \hookrightarrow -\} \mathbf{free}(e) \{\mathbf{emp}\}} \text{FREE}
 \end{array}$$

In **ALLOC**, the program variable x ends up holding the address of the new cell, so $x \hookrightarrow e$ describes that cell. The side condition is needed because e is interpreted in the *final* store, where x has been overwritten.

A full treatment of separation logic has other proof rules similar to Hoare logic. In the examples below we also use the rules for sequencing and consequence, where consequence is stated using entailment ($\Rightarrow$) between heap assertions, and the familiar invariant rule for loops:

$$\frac{\{[b] \star I\} c \{I\}}{\{I\} \text{ while } b \text{ do } c \{[-b] \star I\}} \text{ WHILE}$$

Note how the loop test is combined with the invariant using $\star$: $[b] \star I$ says that b holds and I describes the heap. Using $[b] \wedge I$ instead would be wrong: it would also require an empty heap.

8 Example: Swap

The following program swaps the contents of the cells at p and q :

$$t := *p; u := *q; *p := u; *q := t$$

A natural specification is the following, where a and b are logical variables:

$$\{p \hookrightarrow a \star q \hookrightarrow b\} \text{ swap } \{p \hookrightarrow b \star q \hookrightarrow a\}.$$

We can prove it using the small axioms, framing away the cells (and pure facts) each command does not touch. Each use of FRAME is allowed, since the frame never mentions the assigned variable:

$$\begin{array}{ll} \{p \hookrightarrow a \star q \hookrightarrow b\} & \\ t := *p; & \text{LOAD, framing } q \hookrightarrow b \\ \{[t = a] \star p \hookrightarrow a \star q \hookrightarrow b\} & \\ u := *q; & \text{LOAD, framing } [t = a] \star p \hookrightarrow a \\ \{[t = a] \star [u = b] \star p \hookrightarrow a \star q \hookrightarrow b\} & \\ *p := u; & \text{STORE, framing everything but } p \hookrightarrow a \\ \{[t = a] \star [u = b] \star p \hookrightarrow u \star q \hookrightarrow b\} & \\ *q := t & \text{STORE, framing everything but } q \hookrightarrow b \\ \{[t = a] \star [u = b] \star p \hookrightarrow u \star q \hookrightarrow t\} \Rightarrow & \\ \{p \hookrightarrow b \star q \hookrightarrow a\} & \end{array}$$

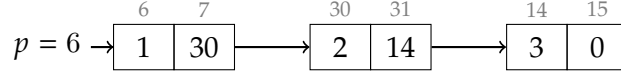
(Before each STORE we also weaken $p \hookrightarrow a$ to $p \hookrightarrow -$, and similarly for q .)

Is this the only specification? No! Because $\star$ forces the two cells to be disjoint, the specification above says nothing about the case where p and q are aliases. In that case the program is the identity, which we can specify as follows:

$$\{[p = q] \star p \hookrightarrow a\} \text{ swap } \{[p = q] \star p \hookrightarrow a\}.$$

9 Linked Lists

Representation. We represent a linked list as a sequence of *nodes*. The node at address p occupies two consecutive cells: its value is stored at p and the address of the next node is stored at $p + 1$. We use 0 as the null pointer; recall that $0 \notin \text{Addr}$, so 0 is never the address of a cell. For example, the following heap holds the list $[1, 2, 3]$ at address 6:



As a finite map, it is $h = \{6 \mapsto 1, 7 \mapsto 30, 30 \mapsto 2, 31 \mapsto 14, 14 \mapsto 3, 15 \mapsto 0\}$. Nodes need not be adjacent or in order; only the pointers matter. (Our `new(e)` allocates a single cell. A real language would allocate a whole node at once, but we will not need allocation below.)

A list predicate. To reason about lists, we define a heap predicate $list(p, L)$, meaning “ p points to a list holding the mathematical list L ,” by recursion on L :

$$\begin{aligned} list(p, []) &\triangleq [p = 0] \\ list(p, v :: L) &\triangleq \exists q. p \hookrightarrow v \star (p + 1) \hookrightarrow q \star list(q, L) \end{aligned}$$

Unfolding the definition step by step:

$$\begin{aligned} &list(6, [1, 2, 3]) \\ \equiv &\exists q_1. 6 \hookrightarrow 1 \star 7 \hookrightarrow q_1 \star list(q_1, [2, 3]) \\ \equiv &\exists q_1, q_2. 6 \hookrightarrow 1 \star 7 \hookrightarrow q_1 \star q_1 \hookrightarrow 2 \star (q_1 + 1) \hookrightarrow q_2 \star list(q_2, [3]) \\ \equiv &\exists q_1, q_2, q_3. 6 \hookrightarrow 1 \star 7 \hookrightarrow q_1 \star q_1 \hookrightarrow 2 \star (q_1 + 1) \hookrightarrow q_2 \star q_2 \hookrightarrow 3 \star (q_2 + 1) \hookrightarrow q_3 \star [q_3 = 0] \end{aligned}$$

The heap above satisfies the last line, with $q_1 = 30$, $q_2 = 14$, and $q_3 = 0$.

What $\star$ rules out. Because each unfolding claims *new* cells that are disjoint from all the others, and the list must end at 0, no list L makes $list(p, L)$ describe a cyclic structure. Similarly, $list(p_1, L_1) \star list(p_2, L_2)$ guarantees that the two lists share no nodes: a shared tail would have to belong to both halves of the heap. In Hoare logic we would need explicit “acyclic” and “disjoint” predicates to say these things. Here $\star$ gives us both for free.

Further Reading

Interested students are referred to two canonical treatments: Reynolds’s [lecture notes](#), or Charguéraud’s [habilitation thesis](#) or textbook *Separation Logic Foundations* (Software Foundations, Volume 6), which this lecture is loosely based on.